

INTRODUCTION TO WEB PROGRAMMING

MODULE 1: JAVASCRIPT ESSENTIAL

JavaScript

- 🔗 An interpreted language
- 🔗 Do not compile a program into machine instructions.
- 🔗 Read and execute instructions by another program

Uses JavaScript Engine

- 🔗 As the interpreter is built into the browser.
- 🔗 `node.js` is an interpreter, but installed independently of browsers as an environment in the computer's operating systems.
- 🔗 allows you to write programs in JavaScript that will turn for example your computer into a server.
- 🔗 Execute source code step by step.
- 🔗 Modern JavaScript Engine uses the `JustIn Time Compilation (JIT Compilation)` which compiles code fragments during the execution of the program (more than a single instruction) and allows to increase performance.
- 🔗 JavaScript is also a high-level language which is relatively easy to understand by people separating us from the hardware details.
- 🔗 JavaScript task is to add dynamics to websites.
- 🔗 JavaScript starts with Netscape Navigator web browser's `LiveScript`

Client-side programming

- 🔗 Code to be executed is loaded together with the page in the browser, on the user's side, and Interpreter which is a part of the web browser allows for its execution.
- 🔗 JavaScript is the sole language supported by all major web browsers, and about 95% of web pages worldwide embedded JavaScript code.
- 🔗 JavaScript offers many frameworks such as `React` or `Angular`.

Server-side programming

- 🔗 JavaScript is also used at the back-end or server-side part of the web applications.
- 🔗 JavaScript programs are executed on the server side, processing data, which after processing will be available on the client-side.
- 🔗 JavaScript flexibility and simplicity makes it appealing in mobile apps and even in programming UAVs.

JavaScript Disadvantages

- 🔗 Limited performance – Not ideal for advanced calculations or high-performance applications.
- 🔗 Limited access – Runs in a sandbox, so it cannot freely access local files and computer resources.
- 🔗 Code is visible – JavaScript code can be viewed and reused by others. `Obfuscation` can make it harder to read.

JavaScript Advantages

- 🔗 Active community – Easy to find help, solutions, and updated tools.
- 🔗 Many libraries and frameworks – Provides ready-to-use features and functions.
- 🔗 Free and accessible – No expensive tools are required; browsers already have JavaScript tools.
- 🔗 Flexible variables – Variables can store different types of data.
- 🔗 Easy to learn – Has a simple and flexible syntax, making it good for beginners.

Development Tools

- 🔗 Code Editor
 - `VS code` - Free for both personal and commercial use
 - `Web Storm` - Commercial use, but has free educational license
 - `Sublime Text` - Require license for both private and commercial
 - `Notepad++` - Free and lightweight
- 🔗 Compiler or Interpreter
- 🔗 Debugger
- 🔗 Development tools run on Windows, MAC, and Linux.

🔗 Development Tools can be: installed Locally, available in the Cloud

Online Development Tools

Also known as Code Playgrounds, are sites that act as a simple editor and runtime environment

Popular JavaScript Programmers

- JSFiddle
- CodePen
- Jsbin
- Plunker

Local Development Environment - Highly recommended to setup one

Other Development tools:

🔗 **Package manager** – enable management libraries (e.g. npm or yarn)

🔗 **Task runners and module bundlers** – automate software development process (e.g. Grunt or Webpack).

🔗 **Testing Framework** – automatic testing (e.g. mocha, jasmine, or jest)

🔗 **Security Analyzer** – used to control the security (e.g. Snyk, RetireJS, or OWASP Dependency Check)

Interpreter

- 🔗 The runtime environment of the program.
- 🔗 It checks for any formal errors.
- 🔗 **node.js** - Server-side application
- 🔗 Web Browser - Chrome or Firefox, Client

Debugger

🔗 A tool that allows you to slow down or even stop the execution of a program, run instructions step-by-step, and look at and analyze the state of the program at any given moment.

🔗 Modern browser are equipped with the developer tools

🔗 Depending on the browser, we may find various tools, but there are certainly:

- The Inspector – analyze individual HTML elements
- The JavaScript console – shows error information and run single JavaScript commands
- The Debugger – shows current values of variables

Enable Developer Tools: Windows and Linux operating systems, all common browsers except Internet Explorer and Edge

Running JavaScript Code

🔗 The **console.log** is treated as a function
🔗 the function is only a log, and console is the object to which the function belongs.

🔗 This type of function, belonging to an object, is generally called a **method Function**

🔗 It is a piece of code that allows you to perform a specific task

🔗 It often take **arguments** (data use during operation).

🔗 we run a function by **calling it**

Two Ways to Execute or Run JavaScript code

using **<script>** Tag

🔗 First, script tags are executed right at the moment when the browser parses the **<script>** tag

🔗 It can be inserted in the page header between the **<head>** and **</head>** tags

🔗 Second, we can insert **<script>** tags in a file (e.g. JavaScript code from different files)

use "src" attribute to point to a separate file that contains the JavaScript code external scripts pointed to by the "src" attribute by using either.

🔗 **defer** – script should be executed after the whole page is loaded

🔗 **async** – script will be executed immediately, but in parallel to parsing the rest of the page.

Steps to Run JavaScript

1. Create a new HTML file (.html) and add the given code.
2. Create another file named main.js.
3. In main.js, write: **console.log("Hello, World!");**
4. Save both files.
5. Open the HTML file in a browser.
6. Open Developer Tools and select the Console.
7. Check the console for "Hello, World!".
8. If using an online environment, put the code in the HTML tab and click Run if needed.

To open the developer console in Google Chrome:

🔗 open the Chrome Menu in the upper-right-hand corner of the browser window and select **More Tools > Developer Tools**.

🔗 You can also use **Option + ⌘ + J** (on macOS), or **Shift + CTRL + J** (on Windows/Linux)

Executing Directly in the Console The Console

🔗 display information, but also allows you to run individual JavaScript instructions.

🔗 Try to open a new tab and type **about:blank** in the address bar. This is a pseudo address that tells your browser to generate and load a minimal blank HTML page.

🔗 Then run the developer tools. At the beginning, we can check what the HTML generated by the browser looks like first tool from the panel, you should see absolutely minimal html code

🔗 After sign **>** or **>>**, enter the instruction that will show "Hello, World!" on the console. Then, press **<enter>** key

MODULE 2: JAVASCRIPT ESSENTIAL

Variables

✳ To declare variables uses keywords **var**, **let** and **const**

✳ JavaScript uses **arithmetic operators** to compute values

✳ JavaScript expression is a combination of values, variables, and operators, which computes to a value. The computation is called an **evaluation**.

Two types of values:

✳ **Fixed Values** - are called **Literals**

✳ **Variables values** – are called **Variables**

Declaring Variables

✳ we declare the variable to reserve a name for it, and memory space is also reserved for the variable.

✳ For the declarations, we use the keywords **var** or **let** for variables and **const** for constants.

✳ **const** – use to declare variables whose values cannot be changed

✳ **let** - prevents us from declaring another variable with the same name

✳ **var** - allows you to re-declare a variable, which can potentially lead to errors in the program execution

Initializing Variables

✳ the variable should be initialized, or given its first value.

✳ Initialization can be done either together with the declaration, or separately as an independent command.

You can assign to a variable:

- a specific value;
- the contents of another variable
- the result returned by a function

Declaration and Strict Mode

✳ The original JavaScript syntax allowed for such negligence, and at the moment of initialization it made this declaration for us.

✳ **"use strict"** - force the interpreter to behave according to modern JavaScript standards.

Variables in the JavaScript language are untyped (or to be more precise, they are weakly and dynamically typed). This means that JavaScript will not control what type of value we store in the variable.

Constant - The **const** keyword is used to declare containers similar to variables. Typical examples of constants are paths to resources, tokens, and other data that never change throughout the lifetime of the script. Constants can also be used as **sub results** in calculations.

Scope

Scope of a variable depends on where it is declared. One of the basic elements that influence the scope of variables is a program block.

Program Blocks

✳ Program blocks are used to separate code into groups of instructions.

✳ Blocks are created using curly brackets.

✳ The instructions inside a block are treated as a separate group.

- ✧ Blocks are used with conditional statements, loops, and functions.
- ✧ A block can be created by grouping a specific range of instructions.
- ✧ Blocks can be nested, meaning one block can be placed inside another block.

let and const

- ✧ If let or const is declared outside a block, it has global scope and can be accessed throughout the program.
- ✧ If declared inside a block, it has local scope. Its name can only be accessed inside that block & blocks nested inside it.

Var

- ✧ The variable declared using keyword var outside the blocks will be global. If you declare it inside a block, it will usually turn out to be global again
- ✧ The problem is that var ignores ordinary program blocks
- ✧ However, if var is declared inside a function, its scope is limited to that function.

Functions

- ✧ To use it, you call the function using its name
- ✧ **function with var** - A var variable declared inside a function has local scope

Variable Shadowing

- ✧ Shadowing happens when a local variable has the same name as a variable in an outer scope.
- ✧ The variable in the more nested scope takes priority over the outer variable.
- ✧ Shadowing also applies to var, but its local scope is based on the function, not an ordinary block.

Hoisting

- ✧ Hoisting is JavaScript's behavior of processing variable and function declarations before the code is executed.
- ✧ It is still good practice to declare variables before using them. Hoisting moves/processes the declaration, not the initialization.
- ✧ let and const behave differently from var when hoisting, so they should also be declared before they are used.

Data Types

Data Type and Type Conversion

- ✧ We can divide the data according to their properties.
- ✧ In JavaScript, data types are divided into primitive and complex.
- ✧ Debugger

✧ **Primitive Data Type** - value stored will be atomic, it will not be possible to extract components from it.

✧ **Complex Data Type** - data consist of many elements of primitive types

✧ **Literal** – a way of writing a specific value directly in code. Almost each data type has its own literal.

The “typeof” operator

- ✧ typeof tells you the data type of a value.
- ✧ It is a unary operator because it works with one argument.
- ✧ The argument can be a literal or variable.
- ✧ It returns a String describing the type.

Primitive Data Type

- ✧ values are atomic, value is one, indivisible element.
- ✧ The six primitive types are: Boolean, Number, BigInt, String, Symbol, undefined null is also a primitive value but is treated as a separate category.

Boolean

- ✧ A logical data type that can only have two values: true/false
- ✧ Used mainly for conditions, control flow, and flags such as enabled/disabled or present/absent.

Number

- ✧ main numeric type. It represents: Integers, Real numbers/fractions
- ✧ For accurate integer calculations, integers should generally stay within: $-(2^{53} - 1)$ to $(2^{53} - 1)$
- ✧ Numbers allow for all typical arithmetic operations. Numbers in JavaScript are usually presented in decimal form
- ✧ JavaScript numbers can be written as:, Hexadecimal: 0x64, Octal: 0o144, Binary: 0b1100100, Exponential: 9e3 → 9000
- ✧ Special Number values: Infinity, -Infinity, NaN

- ✧ NaN means a mathematical/arithmetic operation could not produce a valid number, usually because a value is not a number or cannot be converted to one.

BigInt

- ✧ Used for integers that can be much larger than the normal Number range.
- ✧ BigInt literals use the n suffix.
- ✧ BigInt division is rounded down to the nearest whole number.
- ✧ You cannot directly mix BigInt with other types in arithmetic.
- ✧ BigInt does not have Infinity or NaN.

String

- ✧ A String is a sequence of characters/text.
- ✧ Common operations include: Combining strings, Extracting part of a string, Checking string length
- ✧ Strings are immutable, meaning the original string cannot be directly changed.
- ✧ Use \ as an escape character when needed for quotes inside a string.
- ✧ For -, *, and /, JavaScript tries to convert string values into Numbers. If conversion fails, the result is NaN.
- ✧ + can combine strings into a new string.
- ✧ **String Interpolation** - convenient mechanism that was introduced to JavaScript in 2015. Template strings allow values to be inserted into text using: ``${value}``

Methods and Properties

- ✧ A method is a special function that belongs to an object.

An object can have:

- ✧ Properties - values/data
- ✧ Methods - functions/actions
- ✧ Methods are usually called using dot notation: `object.method()`

- ✧ Primitive values such as Number, BigInt, Boolean, and String have corresponding objects that provide useful methods.
- ✧ **Autoboxing** - When you use a method on a primitive, JavaScript temporarily treats the primitive as its corresponding object.

Commonly used string methods and properties

- ✧ **length**: returns the number of characters in a string;

- ✧ **charAt(index)**: method, returns the character at the "index" position in the string.

- ✧ **slice(beginIndex, endIndex)**: Returns part of a string

- ✧ **split(separator, limit)**: Splits a string and returns an array

undefined

- ✧ undefined is a value that means a variable has no assigned value.
- ✧ Although you can manually assign undefined, the module recommends generally avoiding this and using null when you intentionally want to indicate no meaningful value.

Symbol

- ✧ Symbol is a primitive type introduced in JavaScript in 2015.
- ✧ It has no literal syntax.
- ✧ It is created using a special constructor/function.
- ✧ Each Symbol is guaranteed to be unique. Symbols are mainly used as unique identifiers.

Null

- ✧ null is a primitive value used to indicate that a variable intentionally contains nothing.
- ✧ It is commonly used when a variable is intended to hold a complex value such as an object.
- ✧ One important caveat for null is that when checked with the typeof operator, it will return "object"

Primitive Construction Functions

- ✧ Using literals is not the only way to create primitive values. JavaScript also provides constructor functions. These functions can create/convert primitive values: Boolean(), Number(), BigInt(), String(),
- ✧ These functions are mainly associated with object-oriented programming, but they are also useful for creating or converting primitive values.

When called without arguments:

- String - ""
- Number - 0
- Boolean - false
- BigInt - Requires an initial value.
This can be an integer number that will be converted to a BigInt.

Conversion

JavaScript can convert one data type into another.

- ✧ **Explicit** - done intentionally using functions such as String() or Number().
- ✧ **Implicit** - JavaScript automatically converts values when needed.

Conversion to String

Primitive values can be converted to Strings. String conversion works with primitive types.

Conversion to Number

- ✧ Strings can also contain hexadecimal, octal, or binary numbers using their proper prefixes.
- ✧ An invalid numeric String results in: NaN
- ✧ A BigInt can be converted to Number, but very large BigInt values may lose precision.
- ✧ Boolean true is converted to 1, and false to 0. An attempt to convert an undefined value will result in NaN, while null will be converted to 0.

Conversion to Boolean

- ✧ The value false is always returned for: 0, NaN, empty string, undefined, Null
- ✧ Any other value will result in true being returned.

Conversion to BigInt

- ✧ For BigInt conversion to succeed, the value must represent a number.
- ✧ Accepted forms include: Decimal, Hexadecimal, Octal, Binary
- ✧ Exponential notation is allowed when the argument is a Number.
- ✧ If BigInt conversion fails, JavaScript throws an error and stops the program.

Implicit Conversion

JavaScript can automatically convert values during operations.

- ✧ **Addition with a String** - If one operand is a String, JavaScript converts the other operand to a String.

- ✧ **Subtraction with a String** - Subtraction converts values to Numbers.

Complex Data Types - Complex types: objects and arrays.

Object

- ✧ An object is a complex data type that can be used as a record.
- ✧ A record is a collection of named fields. Each field has a name (key) and a value.
- ✧ These fields are called properties.
- ✧ Objects can store multiple values of different data types in one place.
- ✧ Properties are separated by commas.
- ✧ **Accessing Properties** - Use dot notation. allows you to read or modify a specific property of an object.

Arrays

- ✧ An array is a collection used to store multiple values. Unlike objects, array values normally do not have named keys. Instead, they use indexes.
- ✧ Array indexes start at 0, brackets [], contain different data types and another array (nested array). Nested arrays do not have to have the same length.
- ✧ Array elements can also be objects.

Objects, Arrays, and instanceof

- ✧ In JavaScript, everything except primitives is generally treated as an object.
- ✧ Arrays are a special type of object.
- ✧ typeof does not tell you specifically whether an object is an array. To check whether something is an array, you can use instanceof
- instanceof uses two operands:**
 - ✧ The value being tested
 - ✧ The object/class being checked against

Array length

- ✧ The length property tells you the number of elements in an array.
- ✧ It also counts empty positions in an array.

Array Methods

- ✧ **indexOf()** - Searches for a value.
- ✧ Returns the index if found. Returns -1 if not found.

✧ If there are duplicates, return the index of the first occurrence.

✧ **push()** - Adds an element to the end of the array. Length increases by 1.

✧ **unshift()** - Adds an element to the beginning. Existing elements move right. New element gets index 0.

✧ **pop()** - Removes the last element. Returns the removed element. Array length decreases by 1.

✧ **shift()** - Removes the first element. Returns the removed element. Other elements move left. Array decreases by 1.

✧ **reverse()** - Reverses the order of the array elements.

✧ **slice()** - Creates a new array containing selected elements. It does not change the original array.

- One positive - From that index to the end
- Two positive - From first index up to the second

- Positive + negative - From first index, excluding the specified number of elements at the end

- One negative - Gets the specified number of elements from the end

✧ **concat()** - Creates a new array by combining arrays/elements. It does not change either original array.

Comments

Comments are ignored by the JavaScript interpreter. They are used to explain or document code.

✧ **Single-line comment** - Everything after // on that line is ignored

✧ **Multi-line comment** - Used when the comment needs to cover multiple lines.

Use comments to:

✧ Explain code that is not immediately clear.

✧ Explain intentional unusual behavior.

✧ Document functions and parameters.

✧ Add information such as author, license, or change history.

✧ **Code toggle** - temporarily comments out a piece of code so it will not run.

MODULE 3: JAVASCRIPT ESSENTIAL

Operators

✧ symbols or keywords used to perform actions on values called **operands**.

Unary – works with one operand

Binary – works with two operands

✧ We can differentiate between prefix, postfix, and infix operators.

✧ Operators can also be grouped by their purpose: Assignment, Arithmetic, Logical, Conditional

Assignment Operators

✧ allow for the assigning of values to variables and constants.

✧ The basic assignment operator is the equals sign =

Arithmetic Operators

✧ Perform mathematical operations on numerical values and variables. Except for +, arithmetic operators try to convert values to the Number type before performing the operation.

✧ Use parentheses () to control the order of operations.

✧ basic binary arithmetic operators: addition, subtraction, multiplication, division, remainder, and power*.

Unary Arithmetic Operators

Work with one operand. They convert the operand to the Number type, while the minus operator also negates its value.

Unary increment and decrement Operators

++ Increment – increases a value by 1

-- Decrement – decreases a value by 1

✧ Can be prefix (++x) or postfix (x++).

Compound Assignment Operators

Binary arithmetic operators can be combined with the assignment operator: +=, -=, *=, /=, %=, **=.

Logical Operators

Work with Boolean type values. They work on operands of this type and return values of this type only. Three of such operators

are: && Logical AND, || Logical OR, ! Logical NOT.

Logical Operators and Non- Boolean Values

- ✿ Logical operators can also work with other data types. && and || can return the first or second operand, not necessarily a Boolean value.
- ✿ They use short-circuit evaluation:
- ✿ && returns the first operand if it is false and stops.
- ✿ || returns the first operand if it is true and stops.

Compound Assignment Operators

- ✿ Binary logical operators can be used in combination with the assignment operator
- ✿ &&= Logical AND assignment
- ✿ ||= Logical OR assignment

Examples:

a &&= false → a = a && false

b ||= true → b = b || true

String Operators

The + operator performs string concatenation. If either operand is a String, the other operand is converted to a String and joined to it.

Compound Assignment Operators - You can probably guess that this operator can also be used in conjunction with the replacement operator.

Comparison Operators

- ✿ Used to check equality or inequality. They return true or false.
- ✿ === **Strict equality/identity operator** – values must be equal and have the same type
- ✿ == **Equality** – allows type conversion
- ✿ !== **Strict inequality**
- ✿ != **Inequality**
- ✿ >, <, >=, and <= - They can work with different data types, but are mainly useful for numbers or values that can be converted to numbers. They can also compare strings, but comparing non-numeric strings is more complex and usually not useful.

✿ **Important:** Use === unless you intentionally want comparison between different types.

✿ If an operand is NaN (or has been converted to NaN, e.g. with undefined), equality comparisons return false.

Other Operators

- ✿ **Bitwise operators** – operate on individual bits of values.
- ✿ **delete** – removes a selected property from an object.
- ✿ **? : Ternary operator** – a conditional operator that returns the second or third operand depending on whether the first condition is true or false. The operands can also be expressions that need to be evaluated.

Precedence

- ✿ The JavaScript interpreter uses two operator properties to determine the sequence of operations:
- ✿ **Precedence** determines the priority of operators—higher precedence means the operation is performed first.
- ✿ **Associativity** determines the order of execution when operators have the same precedence.

Interacting with the user

- ✿ Allows the user to enter data, make choices, and confirm options while the program is running.
- ✿ The program can use data provided by the user during execution, even though the data was not known when the program was written or started.
- ✿ The user's input can affect both the data and the execution of the program. This means the program can perform different actions depending on the user's input.
- ✿ For this course, we use dialog boxes to practice basic user interaction.

Dialog Boxes

- ✿ Dialog boxes are popup or modal windows provided by web browsers.
- ✿ When a dialog box is displayed, the user cannot interact with the webpage until the dialog box is closed.

✿ Because they interrupt interaction with the webpage, dialog boxes should not be overused. However, they are useful for learning, displaying important information, or when user input is required.

JavaScript has three main dialog boxes:

1. Alert Dialog Box

✿ The alert() dialog box is the simplest of the three.

✿ It displays a message to the user.

```
alert("Hello!");
```

✿ accepts one optional parameter, which is the text/message displayed.

✿ Important: It only displays information and does not ask the user for input.

2. Confirm Dialog Boxes

✿ The confirm() dialog box allows the user to make a decision.

✿ It has two buttons: OK, Cancel

✿ confirm("Do you want to continue?");

✿ It accepts one optional parameter, which is the message displayed.

✿ The confirm() method returns a Boolean value based on the user's choice: OK → true, Cancel → false

✿ The returned true or false can be used to make the program perform actions conditionally.

3. Prompt Dialog Boxes

✿ The prompt() dialog box allows the user to enter text.

✿ It contains: A message, A single-line text field, OK button, Cancel button

✿ prompt("What is your name?"); - The prompt() method accepts two optional parameters:

✿ **Message** – the text displayed to the user

✿ **Default value** – the text already placed in the input field

✿ Example: prompt("What is your name?", "Hoshi");

✿ If the user clicks OK, the text entered in the field is returned as a String.

✿ If the user clicks Cancel, prompt() returns null.